

ReactiveX

ReactiveX is a library for composing asynchronous and event-based programs by using observable sequences.

It extends the observer pattern (http://en.wikipedia.org/wiki/Observer_pattern) to support sequences of data and/or events and adds operators that allow you to compose sequences together declaratively while abstracting away concerns about things like low-level threading, synchronization, thread-safety, concurrent data structures, and non-blocking I/O.

Observables fill the gap by being the ideal way to access asynchronous sequences of multiple items

	single items	multiple items
synchronous	<code>T getData()</code>	<code>Iterable<T> getData()</code>
asynchronous	<code>Future<T> getData()</code>	<code>Observable<T> getData()</code>

It is sometimes called “functional reactive programming” but this is a misnomer. ReactiveX may be functional, and it may be reactive, but “functional reactive programming” is a different animal. One main point of difference is that functional reactive programming operates on values that change *continuously* over time, while ReactiveX operates on *discrete* values that are emitted over time. (See Conal Elliott’s work for more-precise information on functional reactive programming (<https://github.com/conal/essence-and-origins-of-frp>).)

Why Use Observables?

The ReactiveX Observable model allows you to treat streams of asynchronous events with the same sort of simple, composable operations that you use for collections of data items like arrays. It frees you from tangled webs of callbacks, and thereby makes your code more readable and less prone to bugs.

Observables Are Composable

Techniques like Java Futures

(<http://docs.oracle.com/javase/7/docs/api/java/util/concurrent/Future.html>) are straightforward to use for a single level of asynchronous execution (<https://gist.github.com/4670979>) but they start to add non-trivial complexity (<https://gist.github.com/4671081>) when they’re nested.

It is difficult to use Futures to optimally compose conditional asynchronous execution flows (<https://gist.github.com/4671081#file-futuresb-java-L163>) (or impossible, since latencies of each request vary at runtime). This can be done (<http://www.amazon.com/gp/product/0321349601?ie=UTF8&tag=none0b69&linkCode=as2&camp=1789&creative=9325&creativeASIN=0321349601>), of course. but it quickly becomes complicated (and thus error-prone) or it prematurely blocks on

source, each query becomes complex (and thus error-prone), or prematurely blocks on `Future.get()`, which eliminates the benefit of asynchronous execution.

ReactiveX Observables, on the other hand, are *intended* for composing flows and sequences of asynchronous data (<https://github.com/Netflix/RxJava/wiki/How-To-Use#composition>).

Observables Are Flexible

ReactiveX Observables support not just the emission of single scalar values (as Futures do), but also of sequences of values or even infinite streams. `Observable` is a single abstraction that can be used for any of these use cases. An Observable has all of the flexibility and elegance associated with its mirror-image cousin the `Iterable`.

An Observable is the asynchronous/push “dual”
([http://en.wikipedia.org/wiki/Dual_\(category_theory\)](http://en.wikipedia.org/wiki/Dual_(category_theory))) to the synchronous/pull `Iterable`

event	Iterable (pull)	Observable (push)
retrieve data	<code>T next()</code>	<code>onNext(T)</code>
discover error	<code>throws Exception</code>	<code>onError(Exception)</code>
complete	<code>!hasNext()</code>	<code>onCompleted()</code>

Observables Are Less Opinionated

ReactiveX is not biased toward some particular source of concurrency or asynchronicity. Observables can be implemented using thread-pools, event loops, non-blocking I/O, actors (such as from Akka), or whatever implementation suits your needs, your style, or your expertise. Client code treats all of its interactions with Observables as asynchronous, whether your underlying implementation is blocking or non-blocking and however you choose to implement it.

How is this Observable implemented?

```
public Observable<data> getData();
```

- does it work synchronously on the same thread as the caller?
- does it work asynchronously on a distinct thread?
- does it divide its work over multiple threads that may return data to the caller in any order?
- does it use an Actor (or multiple Actors) instead of a thread pool?
- does it use NIO with an event-loop to do asynchronous network access?
- does it use an event-loop to separate the work thread from the callback thread?

From the Observer’s point of view, it doesn’t matter!

And importantly: with ReactiveX you can later change your mind, and radically change the underlying nature of your Observable implementation, without breaking the consumers of your Observable.

Callbacks Have Their Own Problems

Callbacks solve the problem of premature blocking on `Future.get()` by not allowing anything to block. They are naturally efficient because they execute when the response is ready.

But as with Futures, while callbacks are easy to use with a single level of asynchronous execution, with nested composition they become unwieldy (<https://gist.github.com/4677544>).

ReactiveX Is a Polyglot Implementation

ReactiveX is currently implemented in a variety of languages, in ways that respect those languages' idioms, and more languages are being added at a rapid clip.

Reactive Programming

ReactiveX provides a collection of operators (<documentation/operators.html>) with which you can filter, select, transform, combine, and compose Observables. This allows for efficient execution and composition.

You can think of the Observable class as a “push” equivalent to Iterable (<http://docs.oracle.com/javase/7/docs/api/java/lang/Iterable.html>), which is a “pull.” With an Iterable, the consumer pulls values from the producer and the thread blocks until those values arrive. By contrast, with an Observable the producer pushes values to the consumer whenever values are available. This approach is more flexible, because values can arrive synchronously or asynchronously.

Example code showing how similar high-order functions can be applied to an Iterable and an Observable

Iterable	Observable
<pre>getDataFromLocalMemory() .skip(10) .take(5) .map({ s -> return s + " transformed" }) .forEach({ println "next => " + it })</pre>	<pre>getDataFromNetwork() .skip(10) .take(5) .map({ s -> return s + " transformed" }) .subscribe({ println "onNext => " + it })</pre>

The Observable type adds two missing semantics to the Gang of Four's Observer pattern (http://en.wikipedia.org/wiki/Observer_pattern), to match those that are available in the Iterable type:

1. the ability for the producer to signal to the consumer that there is no more data available (a foreach loop on an Iterable completes and returns normally in such a case; an Observable

"onComplete" or "onError" method)

calls its observer's `onCompleted` method)

2. the ability for the producer to signal to the consumer that an error has occurred (an `Iterable` throws an exception if an error takes place during iteration; an `Observable` calls its observer's `onError` method)

With these additions, `ReactiveX` harmonizes the `Iterable` and `Observable` types. The only difference between them is the direction in which the data flows. This is very important because now any operation you can perform on an `Iterable`, you can also perform on an `Observable`.

DOCUMENTATION

[Observable \(/documentation/observable.html\)](/documentation/observable.html)

[Operators \(/documentation/operators.html\)](/documentation/operators.html)

[Single \(/documentation/single.html\)](/documentation/single.html)

[Subject \(/documentation/subject.html\)](/documentation/subject.html)

[Scheduler \(/documentation/scheduler.html\)](/documentation/scheduler.html)

LANGUAGES

[RxJava !\[\]\(dd161862f9164df98f62b726e9846241_img.jpg\)](https://github.com/ReactiveX/RxJava) (<https://github.com/ReactiveX/RxJava>)

[RxJS !\[\]\(758ebdf4629c903da74c2e079717ae32_img.jpg\)](https://github.com/ReactiveX/rxjs) (<https://github.com/ReactiveX/rxjs>)

[Rx.NET !\[\]\(fe3aebe81acea8d45108cd2768939da7_img.jpg\)](https://github.com/Reactive-Extensions/Rx.NET) (<https://github.com/Reactive-Extensions/Rx.NET>)

[RxScala \(/rxscala\)](/rxscala)

[RxClojure !\[\]\(a8f9309f944226d1420f5fed22e2b6e6_img.jpg\)](https://github.com/ReactiveX/RxClojure) (<https://github.com/ReactiveX/RxClojure>)

[RxSwift !\[\]\(248b91fcdac4810ffd15cf33fb6aec6f_img.jpg\)](https://github.com/ReactiveX/RxSwift) (<https://github.com/ReactiveX/RxSwift>)

[Others \(/languages.html\)](/languages.html)

RESOURCES

[Tutorials \(/tutorials.html\)](/tutorials.html)

COMMUNITY

[GitHub !\[\]\(40770d9ed6ed4f1222ebf89a1396e8b2_img.jpg\)](https://github.com/ReactiveX) (<https://github.com/ReactiveX>)

[Twitter !\[\]\(ccd39a0dc6d5afcc151e1371f9462f58_img.jpg\)](https://twitter.com/ReactiveX) (<https://twitter.com/ReactiveX>)

[Others \(/community.html\)](/community.html)



`ReactiveX  ()` is a collection of open source projects. The content of this page is licensed under Creative Commons Attribution 3.0 License, and code samples are licensed under the BSD License.

